

Group Project: Database For the Construction Submission Portal

The Team



Alexander Beck
CS BS/MS



Tina Cheng
CS Bachelor



Tran Man Tuyen Dinh
Master of CS



Zongyu Mu
Master of CS

Abstract

This project is the design and implementation of a robust Relational Database Management System (RDBMS) application for a centralized Construction Submission Portal. The system aims to modernize the construction permit process for Commercial Mixed-Use Developments by replacing inefficient paper-based workflows with a digital platform. This centralized hub allows design consultants to securely submit building plans and technical documents, enabling complex, parallel reviews by multiple agency bodies to reduce administrative bottlenecks and version control issues. Our technical approach involves engineering a highly normalized database with 20+ relational tables using PostgreSQL, supported by a FastAPI and SQLAlchemy backend. This stack facilitates seamless database integration, handles file management, and enforces data integrity constraints through FastAPI validation. Major milestones accomplished include the setup of the entire tech stack framework (frontend, backend, database). We have successfully normalized the database schema, implemented the tables in PostgreSQL, established basic authentication, and connected all components.

Keywords: Relational Database Management System (RDBMS), Construction Submission Portal, Construction Permit Process, Database Normalization, PostgreSQL, FastAPI, SQLAlchemy, Parallel Agency Review, Version Control, Data Integrity Constraints, Submission Packages, Commercial Mixed-Use Developments.

Overview

The nature of this project falls strictly under the Database Application category. We designed, developed, and implemented a fully functional RDMS database application to power a real-world web application. Specifically, this project is a centralized platform designed to modernize the construction permit process. The core technical focus is engineering a robust relational database that allows design consultants to securely submit building plans - including 3D models and technical documents - to a central hub, enabling a complex workflow where multiple agency bodies can review, issue feedback, and approve the submissions in parallel.

Background

The construction permitting process in many jurisdictions continues to rely on paper-based submissions, fragmented communication, and sequential agency reviews. This traditional workflow often leads to duplicated documentation, lost records, limited transparency, and prolonged approval timelines. According to the National Association of Home Builders (NAHB, 2021), regulatory costs account for approximately 23.8% of the price of a new home in the United States, highlighting the significant economic impact of regulatory inefficiencies. Furthermore, the construction industry remains one of the least digitized major industries globally (McKinsey Global Institute, 2017), despite evidence that digital transformation initiatives have the potential to improve productivity by 14–15% (McKinsey & Company, 2016).

These challenges underscore the need for a centralized and structured digital permit management system. We are targeting the Commercial Mixed-Use Developments (Condos + Retail) projects. These buildings have the most complex permitting requirements. They involve multiple agencies (fire safety, commercial zoning, residential codes), making a centralized RDBMS incredibly valuable for parallel reviews. For example, in real life there are several problems happened during the building permit process that a RDBMS can help to resolve:

- **Administrative Bottlenecks:** Large residential complexes often require sequential reviews from structural, electrical, and plumbing boards. A digitized, parallel review system prevents the plumbing review from being blocked while waiting for the structural paper file to be physically mailed.
- **Version Control Nightmares:** In mixed-use retail/residential builds, floor plans change frequently. Traditional paper workflows lead to agencies reviewing outdated "Version 1" plans while "Version 3" is already drafted. Our system's versioning ensures all Processing Officers view the single source of truth.
- **The Cost of Delay:** As regulatory costs account for 23.8% of a new home's price, delays in high-density condo approvals cost developers thousands of dollars a day in interest. Enforceable data integrity constraints prevent incomplete submissions, reducing the back-and-forth "Written Directions" and speeding up final approval.

In summary, a database-driven solution can support standardized submissions, parallel agency review, real-time status tracking, and enforceable data integrity constraints. By leveraging relational database management principles, the proposed system aims to streamline coordination among stakeholders, reduce administrative delays, and enhance transparency throughout the permitting lifecycle.

How we address the topic

User Requirements

Projects: The database acts as a parent container for the entire building lifecycle by storing project information. Each project has an auto-generated project reference number, project title, Development Type, Building Works, and an Address. Each project contains multiple project members and multiple submission packages. A project must have exactly one active Project Coordinator (PC) and exactly one Developer.

Project Coordinators (PC): The primary administrator who creates the project and assembles the project team. A single PC can manage multiple projects simultaneously. The system records their Name, Role, Professional Registration number, National ID, and email.

Qualified Persons (QP): Licensed professionals assigned to technical scopes. A QP can be assigned to multiple projects, and within a single project, they can be assigned multiple responsibilities or Sections to declare regulatory compliance. The database stores their Name, Role, Registration number, National ID, and email. Their project responsibility data includes their scope of responsibilities for at least one or many agencies, and whether they prepare plans or supervise work. Additionally, a QP can be assigned to manage a specific submission package, taking on the responsibility of the Submission Coordinator (SC).

Developers: The client entity that legally authorizes the project team, authorizes the submission packages. A Developer can own multiple projects and can be either an Organization or an Individual.

- For an Individual, the database records their Name, National ID, and email.
- For an Organization, the database records the Role, Business Registration Number, and a Main acting personnel containing a Name, National ID, and email. The organization can also have up to 2 secondary developer personnel.

Agency Bodies & Staff: The database stores information for agency bodies (e.g., Building Authority) that review submissions and approve it. Each agency body has a Department Head and multiple Processing Officers (PO). Agency staff details include their name, National ID, phone number, and email address. Each PO is expert on one or multiple project types, and their expertise may overlap with that of other POs. This information about PO must also be recorded. After a submission has been submitted, POs will be assigned to each section.

Submission Packages: A specific application instance sent to the government for review and approval. The PC creates a new submission package and assigns a Submission Coordinator (SC). Each submission package belongs to exactly one Project and contains multiple Sections. The database records the submission ID, submission category, submission type, creation timestamp, version number, and current status (Draft, Pending declaration, Pending Authorization, Pending Payment, Submitted, Processing, WD Issued, Approved, Rejected). When a Written Direction (WD) is replied to, the system generates a new version (e.g., Version 2) of the submission to track revisions. Each Submission Package must be managed by exactly one SC. While a package is managed by one SC, an SC can manage multiple submissions.

Sections: Each submission has one or multiple sections. Each section belongs to exactly one Submission Package, targets one Agency, and is assigned to one QP. A section contains multiple documents. The system records the section ID, completion status(Incomplete, Ready for Review, Declared, WD Unlocked), and the version.

Documents: Technical files (like PDF plans, report or 3D models) and other legal documents uploaded for review. Each document belongs to exactly one Section. The database records the document ID, document version, file name, file type, upload timestamp, the User ID of who uploaded it, and a read-only lock flag.

Written Directions (WD): Feedback records created when submissions require corrections. Each WD is issued by one Processing Officer (PO) and is linked to one or multiple specific Sections. The database stores the WD ID, issue timestamp, comment text, and severity/type.

Users' queries/questions

By developing our application, we aim to help users to answer these questions:

1. List all the projects and their current status managed by a specific Project Coordinator.

```
SELECT p.project_id, p.title, p.type, p.address, p.status
FROM project p
WHERE p.project_coordinator_id = :coordinator_national_id
ORDER BY p.status, p.title;
```

→ UI: Projects dashboard — when a Project Coordinator logs in, this query populates their list of projects with status badges (Draft / Ready).

2. List all the submissions and current status of submissions that each Submission Coordinator manages.

```
SELECT s.submission_id, s.version, s.category, s.type,
       s.status, s.creationTime, p.title AS project_title
FROM submission s
JOIN project p ON p.project_id = s.project_id
WHERE s.submission_coordinator_id = :sc_national_id
ORDER BY s.creationTime DESC;
```

→ UI: Submissions page — shows a Submission Coordinator all their active and past submissions grouped by project.

3. Retrieve all pending actionable tasks (e.g., "Pending Declaration", "Pending Payment", "Pending WD Reply") assigned to a specific user (QP or SC) across all active projects.

```

SELECT s.submission_id, s.version, s.status, p.title AS project_title,
       sec.section_id, sec.status AS section_status
FROM submission s
JOIN project p ON p.project_id = s.project_id
JOIN section sec ON sec.submission_id = s.submission_id
                  AND sec.submission_version = s.version
WHERE s.submission_coordinator_id = :user_national_id
      AND s.status IN ('Pending declaration', 'Pending Payment')
ORDER BY s.creationTime ASC;

```

→ UI: Task / notifications panel — alerts the QP or SC to items that require their immediate action.

4. List out all the project members (QPs, Developers) associated with a specific Project ID, including their assigned roles and their current appointment status.

```

SELECT u.name, u.email, i.Role, i.status AS appointment_status
FROM involves i
JOIN user u ON u.national_id = i.national_id
WHERE i.project_id = :project_id
ORDER BY i.Role, u.name;

```

→ UI: Project team page — displays the full member list with role labels and appointment status (Invited / Added / Pending Appointment / Appointed).

Retrieve all Submission Packages associated with a specific Project ID.

```

SELECT s.submission_id, s.version, s.category, s.type,
       s.status, s.creationTime
FROM submission s
WHERE s.project_id = :project_id
ORDER BY s.creationTime DESC;

```

→ UI: Project detail page — lists all submission packages for a project so coordinators can track version history.

6. List all "Draft" submission packages that were created more than 30 days ago.

```

SELECT s.submission_id, s.version, s.creationTime, p.title AS project_title
FROM submission s
JOIN project p ON p.project_id = s.project_id
WHERE s.status = 'Draft'
      AND s.creationTime < NOW() - INTERVAL '30 days'
ORDER BY s.creationTime ASC;

```

→ UI: Admin / coordinator dashboard — surfaces stale draft submissions so coordinators can follow up or discard them.

7. Retrieve the latest version number of a specific Submission Package alongside the current completion status (e.g., Incomplete, Ready for Review, Declared) of every Section it contains.

```

SELECT s.submission_id, s.version, sec.section_id,
       sec.status AS section_status, sec.paid, sec.paymentAmount
FROM submission s
JOIN section sec ON sec.submission_id = s.submission_id
                  AND sec.submission_version = s.version
WHERE s.submission_id = :submission_id
      AND s.version = (
        SELECT MAX(version) FROM submission
        WHERE submission_id = :submission_id
      )
ORDER BY sec.section_id;

```

→ UI: Submission detail page — shows each section's status (Incomplete / Ready for Review / Declared / Approved / Rejected) for the latest version.

8. List out all the submission packages that have received a final "Approved" or "Rejected" status within a specified date range.

```

SELECT s.submission_id, s.version, s.status,
       s.creationTime, p.title AS project_title

```



```

FROM submission s
JOIN project p ON p.project_id = s.project_id
WHERE s.status IN ('Approved', 'Rejected')
      AND s.creationTime BETWEEN :start_date AND :end_date
ORDER BY s.creationTime DESC;

```

→ UI: Reports / history page — lets administrators filter finalized submissions by date range for auditing.

9. List out all the "Processing" submissions currently assigned to a specific agency Processing Officer.

```

SELECT s.submission_id, s.version, s.status,
       p.title AS project_title, sec.section_id
FROM section sec
JOIN submission s ON s.submission_id = sec.submission_id
                  AND s.version = sec.submission_version
JOIN project p ON p.project_id = s.project_id
WHERE sec.assign_to = :processing_officer_national_id
      AND s.status = 'Processing'
ORDER BY s.creationTime ASC;

```

→ UI: Processing Officer dashboard — shows the PO all sections currently assigned to them that are under active processing.

10. List all distinct Agency Bodies involved in the current active Submission Package for a specific Project ID, along with their individual section approval statuses.

```

SELECT sec.target_agency, sec.section_id, sec.status AS section_status
FROM submission s
JOIN section sec ON sec.submission_id = s.submission_id
                  AND sec.submission_version = s.version
WHERE s.project_id = :project_id
      AND s.status NOT IN ('Approved', 'Rejected')
ORDER BY sec.target_agency, sec.section_id;

```

→ UI: Project overview page — shows which government agencies (BCA, MPS, etc.) are involved and the approval status of each of their sections.

11. Retrieve all Written Directions issued for a specific Submission Package, ordered by severity and issue timestamp.

```

SELECT wd.wid, wd.comment_text, wd.severity_type,
       wd.issue_timestamp, pr.section_id
FROM writtendirection wd
JOIN provides pr ON pr.wid = wd.wid
JOIN section sec ON sec.section_id = pr.section_id
WHERE sec.submission_id = :submission_id
ORDER BY wd.severity_type DESC, wd.issue_timestamp ASC;

```

→ UI: Written Directions panel — displays all correction notices for a submission, sorted by severity so the most critical issues appear first.

12. Count the total number of Written Directions issued by each Processing Officer for a specific Project Type within the last quarter.

```

SELECT wd.processing_officer_id, u.name AS officer_name,
       COUNT(wd.wid) AS total_wd_issued
FROM writtendirection wd
JOIN processingofficer po ON po.national_id = wd.processing_officer_id
JOIN expertof eo ON eo.national_id = po.national_id
JOIN user u ON u.national_id = po.national_id
WHERE eo.type = :project_type
      AND wd.issue_timestamp >= NOW() - INTERVAL '3 months'
GROUP BY wd.processing_officer_id, u.name
ORDER BY total_wd_issued DESC;

```

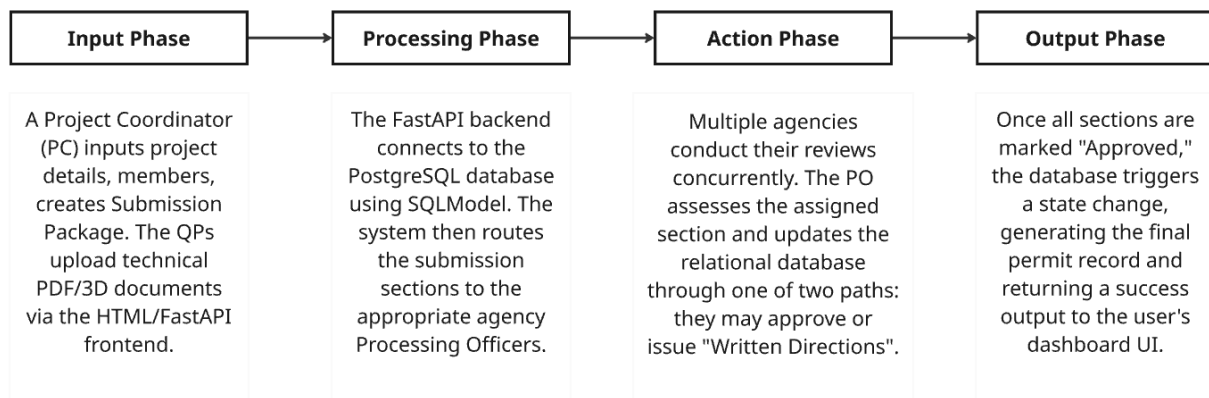
→ UI: Admin analytics page — gives department heads a quarterly summary of how many written directions each PO has issued per project type.

Inputs and outputs to and from our product

Inputs: User profile data, role attributes (Qualified Person/Project Coordinator/Developer), registration numbers, digital construction plans, agency assignment data, reviewer decisions and comments, and review statuses.

Outputs: Database transaction logs, real-time submission status, reviewer comments, automatically generated permit records, and agency notification records.

High-Level Solution Processes



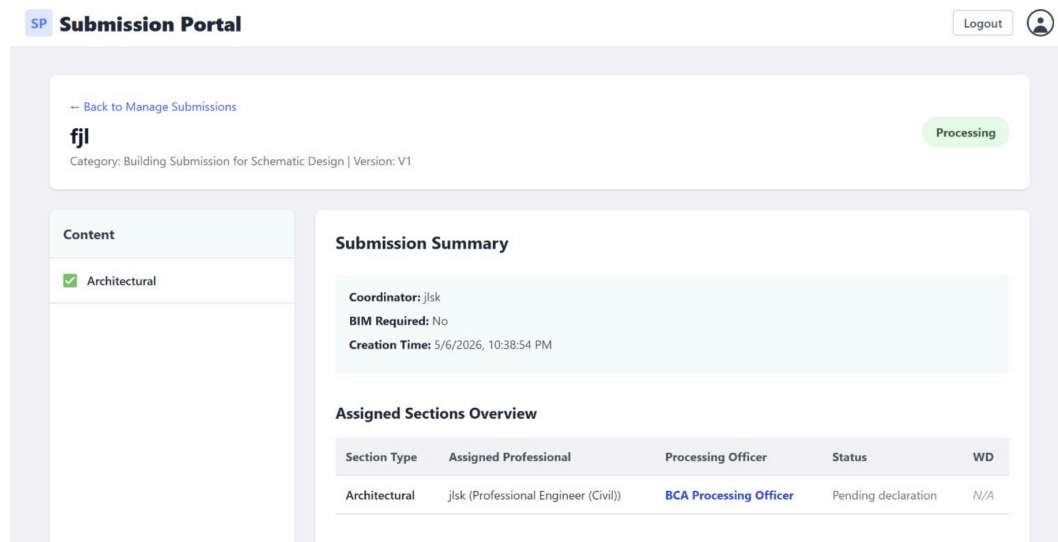
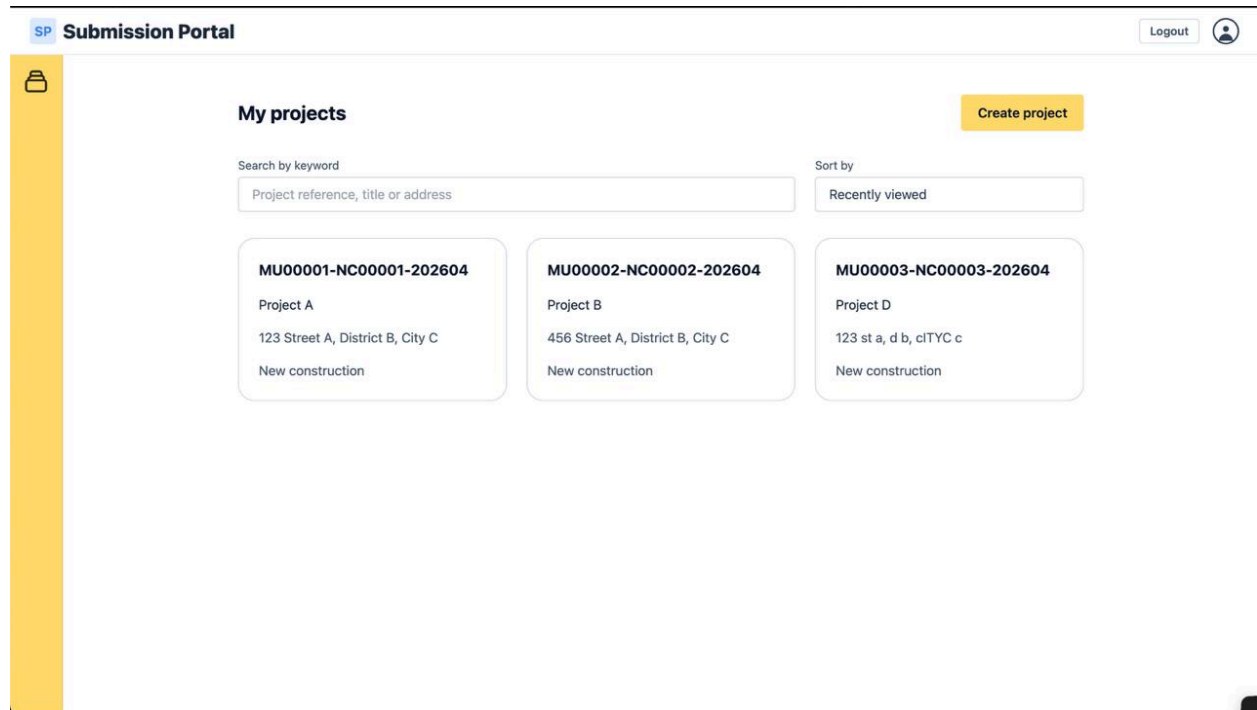
Users' interface design

Below is photos of the UI design. The first is the login screen, followed by a dashboard view of the logged in user's projects. After that is a manage submissions page. Finally, there is an image of the create project page. For more in-depth information about the different functionalities, see the attached demo.

SP Submission Portal

The image shows a login form titled "Login" for the "Submission Portal". The form is centered on a light gray background. It contains the following elements:

- Title:** "Login" in bold black text.
- National ID:** A label above a text input field with the placeholder "Enter your national ID".
- Password:** A label above a text input field with the placeholder "Enter your password".
- Login Button:** A solid blue button with the text "Login" in white.
- Not logged in:** Text centered below the login button.
- Create an account:** A blue link centered below the "Not logged in" text.



SP
Submission Portal
Logout

Project details

Project coordinator
Test

Development type

Select type...

Building works

Select works...

Project title

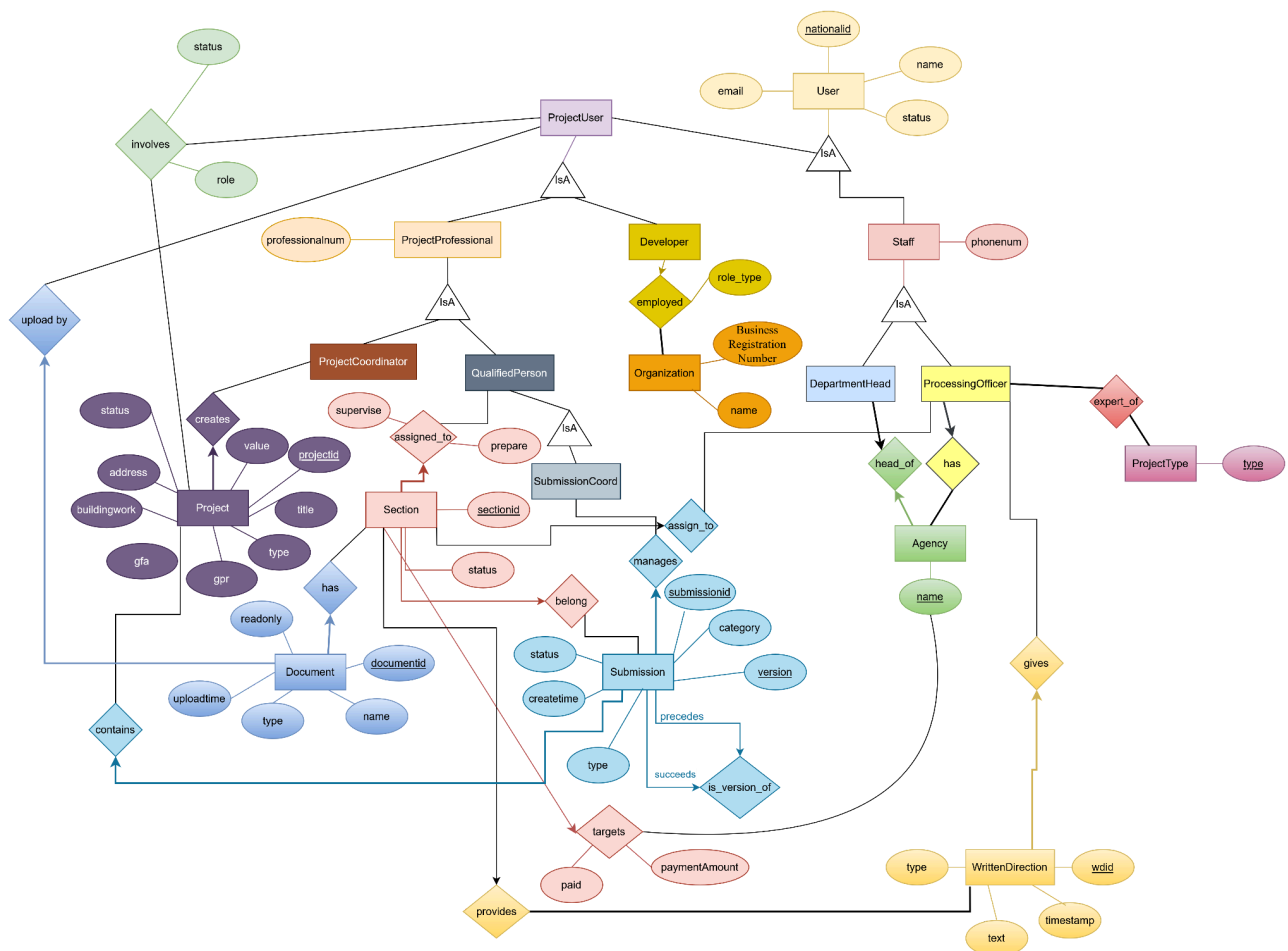
Enter project title...

Cancel

Next

ERD database design

Please refer to the file we attach for better quality.



The following table lists all 21 tables generated from our ERD, including their primary/foreign keys and purpose. These tables were created using SQLAlchemy (Python ORM) and are stored in PostgreSQL.

Table Name	Key Columns	Description
organization	brn (PK), name	Stores construction companies/organizations
user	national_id (PK), name, email, password	Base table for all system users
projectuser	national_id (PK, FK→user)	Users who participate in projects
projectprofessional	national_id (PK, FK→projectuser), professional_id (UNIQUE)	Licensed professionals
developer	national_id (PK, FK→projectuser), employed (FK→organization), role_type	Developers (individual or org)
projectcoordinator	national_id (PK, FK→projectprofessional)	Coordinates projects
qualifiedperson	national_id (PK, FK→projectprofessional)	Qualified persons (QP)
submissioncoord	national_id (PK, FK→qualifiedperson)	Coordinates submissions
staff	national_id (PK, FK→user), phone	Agency staff members
departmenthead	national_id (PK, FK→staff)	Head of a government agency
agency	name (PK), head (FK→departmenthead)	Government agencies (e.g. BCA, MPS)

processingofficer	national_id (PK, FK→staff), agency_name (FK→agency)	Reviews submissions
projecttype	type (PK)	Lookup table: Commercial, Industrial, etc.
expertof	national_id (PK, FK→processingofficer), type (PK, FK→projecttype)	PO's area of expertise
project	project_id (PK), title, type, value, gpr, gfa, building_works, address, project_coordinator_id (FK), status	Construction projects
submission	submission_id (PK), version (PK), category, type, status, creationTime, project_id (FK), submission_coordinator_id (FK), prev_id, prev_version	Formal submissions to agencies
section	section_id (PK), status, assign_to (FK→processingofficer), prepare, supervise, assigned_qp_id (FK), target_agency (FK), paid, paymentAmount, submission_id (FK), submission_version (FK)	Individual sections within a submission
document	document_id (PK), section_id (FK), uploader_user_id (FK), fileType, file_name, file_path, is_read_only, upload_timestamp	Files uploaded to sections
writtendirection	wid (PK), processing_officer_id (FK), comment_text, severity_type, issue_timestamp	Written directions issued by POs
provides	wid (PK, FK→writtendirection), section_id (PK, FK→section)	Links WD to sections
involves	project_id (PK, FK→project), national_id (PK, FK→projectuser), status, Role	Project team membership

Our schema was designed to satisfy BCNF. The table below explains how each normal form is achieved.

Normal Form	Requirement	How Our Schema Satisfies It
1NF	All attributes are atomic; no repeating groups	Every column stores a single atomic value. Roles are split into separate tables (projectcoordinator, qualifiedperson, etc.) rather than stored as a list in one column. The 'involves' table uses a separate row per project-member pair.
2NF	No partial dependencies (non-key attributes depend on the whole PK)	All composite PKs are in junction tables (expertof, involves, provides) where every non-key attribute depends on the full composite key. For example in 'involves', the 'status' and 'Role' fields describe the specific (project_id, national_id) pair — not just one side.
3NF	No transitive dependencies (non-key attributes depend only on the PK)	Organization details are stored in the 'organization' table and referenced by FK in 'developer'. Agency details are in the 'agency' table referenced from 'processingofficer'. No non-key column derives its value from another non-key column.
BCNF	Every functional determinant is a candidate key	All tables have a single candidate key (either a single PK or a well-defined composite PK). For example, 'user.email' is declared UNIQUE — if it were a determinant it would be promoted to a candidate key. The inheritance chain (user → projectuser → projectprofessional → ...) ensures each table is determined solely by its own PK.

Functional Dependencies (FDs)

Representative FDs in our schema:

- national_id → name, email, password (user table)
- professional_id → national_id (projectprofessional, via UNIQUE constraint)
- brn → name (organization table)
- agency.name → head (agency table)
- (project_id, national_id) → status, Role (involves table)
- (wid, section_id) → [no non-key attrs] (provides — pure junction)

None of the above FDs violate BCNF because in every case the determinant is a candidate key of its table. No lossless-join decomposition was required since the schema was designed in BCNF from the start.

[illegible]

ID	Test Case	Input Value	Expected Output	Pass/Fail	Date Tested
1	Hash password	"123456"	Hashed string (SHA-256)	Pass	Mar 29, 2026
2	Verify password – correct	correct pw + hash	True	Pass	Mar 29, 2026
3	Verify password – wrong	wrong pw + hash	False	Pass	Mar 29, 2026
4	API root (GET /)	HTTP GET to /	200 OK, body contains "live"	Pass	Mar 29, 2026

5	Get all users (GET /users/)	HTTP GET to /users/	200 OK	Pass	Mar 29, 2026
6	Database connection	engine.connect()	Connection object not None	Pass	May 1, 2026
7	Insert a valid user record	email = "user1@gmail.com", role = "qp"	Record inserted successfully	Pass	May 1, 2026
8	Insert duplicate email	email = "user1@gmail.com" (already exists)	UNIQUE constraint violation	Pass	May 1, 2026
9	Insert user with NULL email	email = NULL	NOT NULL constraint violation	Pass	May 1, 2026
10	Insert valid submission package	projectid = 101, PC nationalid = 1, SC nationalid = 5	Submission created, status = "Draft"	Pass	May 1, 2026
11	Insert submission with invalid nationalid	PC nationalid not exist	FOREIGN KEY violation	Pass	May 1, 2026
12	Assign ProcessingOfficer to Section	submissionid = 55, sectionid = 3, PO nationalid = 7	Assignment created successfully	Pass	May 1, 2026
13	Assign PO with invalid submissionid	submissionid = 8888 (not exist)	FOREIGN KEY violation	Pass	May 1, 2026
14	Update Section status to "Approved"	section_status = "Approved"	Status updated successfully	Pass	May 1, 2026
15	Insert invalid submission_status	"InvalidStatus"	CHECK constraint violation	Pass	May 1, 2026
16	Generate permit after full approval	All Sections = "Approved"	Permit created & linked	Pass	May 2, 2026
17	Generate permit before approval	Some Sections = "Pending"	Permit NOT generated	Pass	May 2, 2026

18	Delete submission with existing Sections	submission referenced	Deletion restricted	Pass	May 2, 2026
19	Insert user with invalid role	role_type = "admin"	CHECK constraint violation	Pass	May 2, 2026
20	Insert QualifiedPerson without professionalNum	professionalNum = NULL	NOT NULL constraint violation	Pass	May 2, 2026
21	Insert Organization without BRN	business_registration_number = NULL	NOT NULL constraint violation	Pass	May 2, 2026
22	QP submits without SC role	nationalid = 10 (not SC)	Permission error	Pass	May 2, 2026

Tech stack - Programming/database libraries to be used

Frontend

Due to the nature of the course, the majority of the efforts for this project were put into the database itself. However, other aspects were worked on as well, such as the frontend and file management system. The frontend was written using basic HTML and CSS, and little time was allocated to it. Tailwind was also added to make the implementation of CSS easier (Tailwind Labs, N.D.).

The main component that were included in the frontend are as follows:

1. A table for viewing the output
2. A button to upload files
3. Login page

Backend

The backend is more sophisticated than the frontend. It is responsible for allowing the user to upload and download files, have login capabilities, and connect to the database.

To handle these features, we used FastAPI (Ramírez, S, *FastAPI*, N.D.). FastAPI is a Python library that allows for easy file sharing, API endpoints, and integration with a SQL database (with SQLAlchemy). Best of all, it is built with the mindset of type hints in mind, which makes it easy to debug. Login management was handled by FastAPI, as the documentation includes multiple examples on authentication (Ramírez, S, *FastAPI*, N.D.). Furthermore, another tool by the creators of FastAPI, SQLAlchemy, works seamlessly with FastAPI to create database connections (Ramírez, S, *SQLModel*, N.D.).

Database

For our database, we decided to use PostgreSQL, as it is a common choice in the industry (PostgreSQL Global Development Group, N.D.).

Supplementary Materials

In addition to the above listed sources for the project, we followed some tutorials and guides to allow us to focus our attention on the DBMS portion of the project. One such tutorial was found on Medium, and it describes how to connect Python to Postgres using FastAPI (Kim, K, 2023). Another document shows how to handle uploading files through FastAPI (Ulili, S, 2025).

How we document our program codes

We documented our code by focusing on readability first, using clear and descriptive names for all tables, columns, and variables so the code naturally explains what it is doing. We rely on high-level external documentation, such as our ERD and a project README, to provide a clear overview of the database structure and system architecture before anyone even looks at the raw code.

List of Problem

1. Because we use sqlalchemy to make the table for our database, it was difficult to integrate triggers and constraint in pure PostgreSQL
2. Setting up async PostgreSQL sessions with SQLAlchemy was challenging because the sync engine caused conflicts with FastAPI's async request lifecycle, requiring a full refactor of the session dependency injection pattern.
3. Normalizing a database was much more difficult than we expected for a database with 20 relational tables.
4. Implementing JWT token expiry and refresh logic was complex — we needed to handle token expiration checks server-side, return proper HTTP 401 responses, and ensure the frontend was redirected correctly on expiry.
5. File upload validation was problematic without proper checks. Without enforcing MIME type and file size limits early in the request pipeline, invalid files could reach storage, causing downstream errors.

Lessons Learned

1. We can do triggers and constraint by writing validation through fastapi
2. SQLAlchemy's inheritance system requires explicit table=True flags on each subtype model; relying on inheritance alone causes ORM mapping conflicts that are difficult to debug without understanding how SQLAlchemy maps Python class hierarchies to relational tables.
3. Split up the normalization process instead of doing it all at once.
4. Designing 20+ normalized tables from the start prevented data redundancy but required careful planning of FK relationships and inheritance hierarchies.
5. Learned to leverage Pydantic type hints through SQLAlchemy for both DB models and API schemas simultaneously, reducing boilerplate significantly.
6. Implementing stateless JWT auth taught the team about token lifecycle, refresh strategies, and secure password hashing with bcrypt.
7. Transitioning to async database sessions required refactoring dependency injection patterns and understanding FastAPI lifespan context managers.
8. Established feature-branch workflows with pull request reviews to coordinate across 4 team members without merge conflicts.

Conclusion

We have finished the setup for the backend, frontend, and database framework, normalized the database, and implemented the tables in PostgreSQL schema. We have also defined the constraints/triggers through fastapi. Connect the backend to the database, and the backend to the frontend. We also implement basic authentication and

the frontend and backend logic for login and registering accounts. We have also made the project table view and the landing page.

We were a bit behind our original timeline as we were not able to implement project team features and create project button form because of the complexity of the setup, it pushed back everything on the timeline. Despite this setback, we managed to complete the project in a timely manner.

Timeline

Task	Wk 1 (Mar 30)	Wk 2 (Apr 6)	Wk 3 (Apr 13)	Wk 4 (Apr 20)	Wk 5 (Apr 27)
Finalize ERD & Schema					
Complete Backend CRUD					
Frontend Pages (Login/Dashboard)					
File Upload Integration					
Agency Review Workflow					
API Integration Tests					
End-to-End Testing					
Final Demo & Submission					

Deliverable

The final deliverable will be a demo created based on ERD and database solutions. We will evaluate our final deliverable by ensuring we meet all the user requirements, pass all the test cases, and ensure the usability of the application by testing all features of the application.

Demo Video: <https://drive.google.com/file/d/1WlctfpetQXYHjnu0fuDp0-BqloMpw3Xz/view?usp=sharing>

Future Work

There are many directions that this application can take to improve. One would be the addition of push notifications, such as through email, that would notify stakeholders on status changes and WD issuance. Another would be to change the frontend from basic HTML/JS to a more modern framework, like React or Angular. Additionally, adding a feature for displaying metrics about processing time or bottleneck identification would make it more functional in the real world. There could also be third party integration for allowing easier sign in (such as using government credentials).

References

Ataide, M., Braholli, O., & Siegele, D. (2023). Digital transformation of building permits: Current status, maturity, and future prospects. *Buildings*, 13(10), 2554. <https://doi.org/10.3390/buildings13102554>

Kim, K. (2023). *A Guide to Connecting PostgreSQL and Pythons' Fast API: From Installation to Integration*. <https://medium.com/@kevinkoech265/a-guide-to-connecting-postgresql-and-pythons-fast-api-from-installation-to-integration-825f875f9f7d>

McKinsey Global Institute. (2017). *Digital America: A Tale of the Haves and Have-Mores*. <https://www.mckinsey.com/mgi/overview/in-the-news/digital-america-a-tale-of-the-haves-and-have-mores>

McKinsey & Company. (2016). *Imagining construction's digital future*. <https://www.mckinsey.com/capabilities/operations/our-insights/imagining-constructions-digital-future>

National Association of Home Builders. (2021). *Government regulation in the price of a new home*. <https://www.nahb.org/news-and-economics/housing-economics-plus/special-studies/special-studies-pages/government-regulation-in-price-of-new-home-in-2021>

Pablo. (2025). *How to Integrate 3D Models into Your Website Using Only HTML and CSS*. <https://dev.to/checode/how-to-integrate-3d-models-into-your-website-using-only-html-and-css-lok>

PostgreSQL Global Development Group. PostgreSQL [Computer software]. <https://www.postgresql.org/>

Ramírez, S. FastAPI [Computer software]. <https://github.com/fastapi/fastapi>

Ramírez, S. SQLAlchemy [Computer software]. <https://github.com/fastapi/sqlmodel>

Tailwind Labs. Tailwind CSS [Computer software]. <https://github.com/tailwindlabs/tailwindcss>

Ulili, S. (2025). *Uploading Files Using FastAPI: A Complete Guide to Secure File Handling*. <https://betterstack.com/community/guides/scaling-python/uploading-files-using-fastapi/>